Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase V · Alineamiento, RL, agentes e interpretabilidad

Módulo 21

Aprendizaje por refuerzo y preferencias

Gradiente de política, PPO, modelos de recompensa, DPO y GRPO

Resumen del módulo

Edición de octubre de 2026

Sobre este resumen

Este documento reúne lo indispensable de los apuntes del Módulo 21: el texto expositivo, todas las definiciones y todos los resultados (teoremas, proposiciones, lemas y corolarios), las tablas y las observaciones de contenido. Se han suprimido las demostraciones, los ejemplos, las lecturas, las figuras, los ejercicios y sus soluciones. La numeración de temas, secciones, enunciados y ecuaciones es la misma que en los apuntes, de modo que cualquier referencia puede buscarse allí; las que remiten a elementos suprimidos (un ejemplo, una figura, una ecuación de una demostración) conservan su número en los apuntes.

Notación

Símbolo	Significado
$\mathcal{S}$, $\mathcal{A}$, $p(s' \mid s, a)$	estados, acciones y probabilidades de transición de un MDP
$\gamma \in [0,1]$, G_t	factor de descuento y retorno desde el instante t
$\tau = (s_0, a_0, r_1, s_1, \dots)$	trayectoria o episodio
$\pi_\theta(a \mid s)$	política con parámetros θ
V^π, Q^π, $A^\pi = Q^\pi - V^\pi$	funciones de valor de estado, de acción y ventaja
$J(\theta)$	rendimiento esperado de la política π_θ
x, $y = y_{1:T}$, $\mathcal{V}$	instrucción (*prompt*), respuesta de T tokens y vocabulario
$r(x, y)$, r_ϕ	recompensa de una respuesta y modelo de recompensa con parámetros ϕ
π_{ref}, π_{old}	política de referencia (congelada) y política que generó las muestras
$\rho = \pi_\theta / \pi_{\text{old}}$, ε	cociente de importancia y parámetro de recorte de PPO
$\beta > 0$, $Z(x)$	intensidad de la penalización KL y función de partición
$D_{\text{KL}}(p \,\|\, q)$	divergencia de Kullback–Leibler, en nats
$y_w \succ y_l$	la respuesta y_w se prefiere a y_l
$\sigma(z) = 1/(1 + e^{-z})$	función sigmoide
e_a	vector canónico (*one-hot*) de la acción a

Índice

Sobre este resumen **ii**

1 Procesos de decisión de Markov y el LLM como política **1**
1.1 La interfaz agente-entorno . . . 1
1.2 Recompensa y retorno . . . 2
1.3 Funciones de valor y ecuación de Bellman . . . 2
1.4 El modelo de lenguaje como política . . . 2

2 Gradiente de política **4**
2.1 Políticas parametrizadas y función de puntuación . . . 4
2.2 El teorema del gradiente de política . . . 4
2.3 REINFORCE . . . 5
2.4 Causalidad y recompensa futura . . . 5
2.5 Línea base y varianza . . . 5
2.6 Ventaja, crítico y estimación generalizada de la ventaja . . . 6

3 Optimización proximal y regularización KL **7**
3.1 Muestreo de importancia y objetivo sustituto . . . 7
3.2 El objetivo recortado de PPO . . . 7
3.3 Penalización KL respecto a un modelo de referencia . . . 8
3.4 La política óptima en forma cerrada . . . 9

4 Preferencias, modelo de recompensa y RLHF **10**
4.1 El modelo de Bradley–Terry . . . 10
4.2 Estimación por máxima verosimilitud . . . 11
4.3 Modelos de recompensa neuronales . . . 11
4.4 El procedimiento RLHF de InstructGPT . . . 11

5 Optimización directa de preferencias (DPO) **13**
5.1 De la política óptima a la recompensa . . . 13
5.2 Clases de equivalencia de recompensas . . . 13
5.3 La pérdida DPO . . . 14
5.4 DPO frente a RLHF con PPO . . . 14

6 Retroalimentación de IA, recompensas verificables y sobreoptimización 16
6.1 Constitutional AI y retroalimentación de IA 16
6.2 Recompensas verificables 17
6.3 GRPO: optimización de política relativa al grupo 17
6.4 Sobreoptimización de la recompensa 18

Bibliografía 20

Tema 1

Procesos de decisión de Markov y el LLM como política

Este tema introduce el lenguaje del aprendizaje por refuerzo: agente, entorno, recompensa, retorno, política y funciones de valor. La presentación sigue el capítulo 3 de Sutton y Barto [22] y la clase 2 del curso de David Silver [20]; la última sección traslada el formalismo a la generación de texto, como en la clase 17 de CS336 [7].

1.1 La interfaz agente-entorno

En cada instante $t = 0, 1, 2, \dots$ un *agente* observa el estado S_t del *entorno*, elige una acción A_t y recibe, en el instante siguiente, una recompensa numérica R_{t+1} y un nuevo estado S_{t+1}. La sucesión resultante

$$S_0,\, A_0,\, R_1,\, S_1,\, A_1,\, R_2,\, S_2,\, \dots$$

se llama *trayectoria*. Cuando la interacción termina en un estado terminal tras un número finito de pasos, la trayectoria se llama *episodio*.

Definición 1.1 (Proceso de decisión de Markov finito). Un *proceso de decisión de Markov* (MDP) finito es una tupla $(\mathcal{S}, \mathcal{A}, p, \rho_0, \gamma)$ formada por un conjunto finito de estados $\mathcal{S}$, un conjunto finito de acciones $\mathcal{A}$, una *dinámica*

$$p(s', r \mid s, a) = P\big(S_{t+1} = s',\, R_{t+1} = r \mid S_t = s,\, A_t = a\big),$$

que para cada par (s, a) es una distribución sobre $\mathcal{S} \times \mathbb{R}$ con soporte finito, una distribución inicial ρ_0 sobre $\mathcal{S}$ y un *factor de descuento* $\gamma \in [0, 1]$. La palabra «Markov» expresa que la distribución de (S_{t+1}, R_{t+1}) depende de la historia solo a través de (S_t, A_t).

Definición 1.2 (Política). Una *política* (estacionaria y estocástica) es una aplicación que asigna a cada estado s una distribución $\pi(\cdot \mid s)$ sobre $\mathcal{A}$. Una política *parametrizada* π_θ depende de un vector de parámetros $\theta \in \mathbb{R}^d$.

Fijados un MDP y una política, la probabilidad de un prefijo de trayectoria $\tau = (s_0, a_0, r_1, s_1, \dots, s_T)$ se obtiene por la regla de la cadena (módulo 1):

$$p_\pi(\tau) = \rho_0(s_0) \prod_{t=0}^{T-1} \pi(a_t \mid s_t)\, p(s_{t+1}, r_{t+1} \mid s_t, a_t). \tag{1.1}$$

Esta factorización separa lo que controla el agente (los factores π) de lo que impone el entorno (los factores p), y es la base del teorema del gradiente de política del tema 2.

1.2 Recompensa y retorno

El objetivo del agente no es maximizar la recompensa inmediata sino una cantidad acumulada [22, cap. 3].

Definición 1.3 (Retorno). El *retorno* desde el instante t es

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

En una tarea episódica de horizonte T se toma $R_k = 0$ para $k > T$, de modo que la suma es finita y se admite $\gamma = 1$.

Proposición 1.4. *(i) Para todo t, $G_t = R_{t+1} + \gamma\, G_{t+1}$.*

(ii) Si $|R_k| \le R_{\max}$ para todo k y $\gamma < 1$, entonces $|G_t| \le R_{\max}/(1-\gamma)$.

La recursión (i) permite calcular todos los retornos de un episodio en una sola pasada hacia atrás, empezando por $G_T = 0$ (ejercicio 1.3).

1.3 Funciones de valor y ecuación de Bellman

Definición 1.5 (Funciones de valor y ventaja). Sea π una política. La *función de valor de estado* y la *función de valor de acción* son

$$V^\pi(s) = \mathbb{E}_\pi\big[G_t \mid S_t = s\big], \qquad Q^\pi(s,a) = \mathbb{E}_\pi\big[G_t \mid S_t = s,\, A_t = a\big],$$

donde $\mathbb{E}_\pi$ indica que las acciones futuras se eligen con π. La *función ventaja* es $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$.

La ventaja mide cuánto mejor (o peor) que la media de la propia política es elegir a en s y continuar después con π. Es la cantidad que pondera los gradientes de todos los métodos de los temas 2, 3 y 6.

Teorema 1.6 (Ecuación de Bellman para V^π). *Para todo estado s,*

$$V^\pi(s) = \sum_a \pi(a \mid s)\, Q^\pi(s,a), \qquad Q^\pi(s,a) = \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V^\pi(s')\big].$$

En particular, $V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V^\pi(s')\big]$.

Corolario 1.7. *Para todo estado s, $\displaystyle\sum_a \pi(a \mid s)\, A^\pi(s,a) = 0$.*

La ecuación de Bellman es un sistema lineal de $|\mathcal{S}|$ ecuaciones con $|\mathcal{S}|$ incógnitas. Para $\gamma < 1$ tiene solución única (ejercicio 1.6); el capítulo 4 de [22] la resuelve de forma iterativa.

1.4 El modelo de lenguaje como política

La generación autorregresiva encaja en la definición 1.1 con una dinámica especialmente sencilla [7].

Definición 1.8 (MDP de tokens). Sean $\mathcal{V}$ un vocabulario con un símbolo de fin y x una instrucción. El *MDP de tokens* asociado a x tiene como estados las secuencias $s_t = (x, y_1, \ldots, y_t)$, como acciones los tokens $a_t = y_{t+1} \in \mathcal{V}$, una transición determinista $s_{t+1} = (s_t, a_t)$ (concatenación), estado inicial $s_0 = x$ y estados terminales los que acaban en el símbolo de fin o alcanzan una longitud máxima. La recompensa es nula salvo en la transición final, en la que vale $r(x, y)$ para la respuesta completa $y = y_{1:T}$, y se toma $\gamma = 1$.

Proposición 1.9. *En el MDP de tokens, con una política $\pi_\theta(y_{t+1} \mid x, y_{1:t})$:*

(i) la probabilidad de una trayectoria coincide con la de la respuesta, $p_{\pi_\theta}(\tau) = \pi_\theta(y \mid x) = \prod_{t=1}^{T} \pi_\theta(y_t \mid x, y_{<t})$;

(ii) el retorno desde cualquier instante es $G_t = r(x, y)$, y $V^{\pi_\theta}(x) = \mathbb{E}_{y \sim \pi_\theta(\cdot|x)}[r(x, y)]$;

(iii) para un prefijo s_t y un token a, $Q^{\pi_\theta}(s_t, a) = V^{\pi_\theta}(s_t a)$, donde $s_t a$ denota la concatenación.

Elemento del MDP	En un modelo de lenguaje
Estado s_t	instrucción x más los tokens generados $y_{1:t}$
Acción a_t	siguiente token y_{t+1}; $\lvert\mathcal{A}\rvert = \lvert\mathcal{V}\rvert$, del orden de 10^4–10^5
Transición	determinista y conocida: concatenar el token
Política $\pi_\theta(a \mid s)$	softmax de los logits del Transformer (módulo 14)
Recompensa	normalmente solo al final: $r(x, y)$
Episodio	una respuesta completa, hasta el símbolo de fin

Cuadro 1.1: Correspondencia entre los elementos de un MDP y la generación de texto.

Observación 1.10 (Bandido contextual). Como la transición es conocida y la recompensa llega solo al final, la respuesta entera y puede verse como una única acción tomada en el contexto x con probabilidad $\pi_\theta(y \mid x)$. Este caso particular de MDP de un solo paso se llama *bandido contextual*. Los métodos de preferencias de los temas 4 y 5 trabajan a este nivel; PPO (tema 3) trabaja token a token con ventajas $A^\pi(s_t, a_t)$. El número de acciones del bandido es enorme: con $|\mathcal{V}| = 5 \cdot 10^4$ y $T = 100$ hay $|\mathcal{V}|^T \approx 10^{470}$ respuestas, por lo que ninguna suma sobre y puede calcularse explícitamente.

Tema 2

Gradiente de política

Este tema deduce el gradiente del rendimiento esperado respecto a los parámetros de la política y lo convierte en un algoritmo estimable por muestreo, REINFORCE [23]. Después estudia la varianza del estimador y las dos técnicas que la reducen: la línea base y la ventaja estimada por un crítico, incluida la estimación generalizada de la ventaja (GAE) [17]. Se siguen las §13.1–13.5 de Sutton y Barto [22] y las clases 5 y 6 de CS285 [12].

2.1 Políticas parametrizadas y función de puntuación

Definición 2.1 (Rendimiento y función de puntuación). Sea π_θ una política parametrizada en un MDP episódico. El *rendimiento* es $J(\theta) = \mathbb{E}_{\tau\sim p_\theta}[R(\tau)]$, con $R(\tau) = \sum_{t=0}^{T-1} \gamma^t r_{t+1}$ y $p_\theta = p_{\pi_\theta}$ dada por (1.1). Si $\pi_\theta(a \mid s) > 0$, el vector $\nabla_\theta \log \pi_\theta(a \mid s)$ se llama *función de puntuación* (*score*) o vector de elegibilidad.

La parametrización más común en espacios de acciones discretos es la softmax de unas preferencias o logits [22, §13.1]. En un LLM los logits son la salida del Transformer y dependen de θ de forma no lineal; el caso tabular, en el que los propios logits son los parámetros, permite calcular todo a mano.

Proposición 2.2 (Puntuación de la softmax). *Sea* $\pi_\theta = \mathrm{softmax}(\theta)$, *con* $\theta \in \mathbb{R}^K$ *y* $\pi_\theta(a) = e^{\theta_a} / \sum_b e^{\theta_b}$. *Entonces*

$$\nabla_\theta \log \pi_\theta(a) = e_a - \pi_\theta.$$

Si los logits son $z = f_\theta(s)$, *por la regla de la cadena* $\nabla_\theta \log \pi_\theta(a \mid s) = J_f(s)^\top (e_a - \pi_\theta(\cdot \mid s))$, *con* J_f *la matriz jacobiana de* f_θ.

El vector $e_a - \pi_\theta$ es exactamente el gradiente, cambiado de signo, de la entropía cruzada con etiqueta a (módulo 8). Esta coincidencia explica la relación entre gradiente de política y ajuste supervisado que se establece más abajo.

2.2 El teorema del gradiente de política

Lema 2.3 (Truco de la derivada logarítmica). *Si* $p_\theta(z) > 0$ *es diferenciable en* θ, *entonces* $\nabla_\theta p_\theta(z) = p_\theta(z)\, \nabla_\theta \log p_\theta(z)$.

Teorema 2.4 (Gradiente de política, forma de trayectorias). *En un MDP episódico finito con política diferenciable y estrictamente positiva,*

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim p_\theta}\Big[R(\tau) \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\Big].$$

Lo notable del teorema es que la dinámica p desaparece del gradiente: basta con poder *muestrear* trayectorias y evaluar su recompensa, aunque ni el entorno ni la recompensa sean derivables. En la forma de Sutton y Barto [22, §13.2], el teorema se enuncia con la distribución de estados de la política y Q^π; la versión anterior es la de la clase 5 de CS285 [12].

Corolario 2.5 (Gradiente de política en un LLM). *En el MDP de tokens (definición 1.8),*

$$\nabla_\theta J(\theta) = \mathbb{E}_x \, \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}\Big[r(x,y)\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t})\Big].$$

El sumatorio $\sum_t \nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t})$ es el gradiente de la log-verosimilitud de y, el mismo que maximiza el SFT del módulo 20. El gradiente de política es, por tanto, un SFT sobre las propias muestras del modelo, ponderado por su recompensa [7].

2.3 REINFORCE

Definición 2.6 (Estimador REINFORCE). Dadas N trayectorias independientes $\tau^{(1)}, \ldots, \tau^{(N)} \sim p_\theta$, el estimador REINFORCE del gradiente es

$$\widehat{g} = \frac{1}{N}\sum_{i=1}^{N} R(\tau^{(i)}) \sum_t \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}),$$

y el algoritmo actualiza $\theta \leftarrow \theta + \eta\, \widehat{g}$ con una tasa de aprendizaje $\eta > 0$.

Por el teorema 2.4, $\mathbb{E}[\widehat{g}] = \nabla_\theta J$: el estimador es insesgado. Williams [23] lo introdujo en 1992 como un caso de una familia de algoritmos que siguen el gradiente en esperanza.

2.4 Causalidad y recompensa futura

Proposición 2.8 (Las recompensas pasadas no aportan). *Si $t' < t$, entonces $\mathbb{E}_{p_\theta}\big[r_{t'+1}\nabla_\theta \log \pi_\theta(a_t \mid s_t)\big] = 0$. En consecuencia,*

$$\nabla_\theta J(\theta) = \mathbb{E}_{p_\theta}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, \widehat{Q}_t\Big], \qquad \widehat{Q}_t = \sum_{t'\geq t} \gamma^{t'} r_{t'+1}.$$

La cantidad $\widehat{Q}_t$ (la «recompensa futura», *reward-to-go*) es una estimación de una muestra de $Q^\pi(s_t, a_t)$, salvo el factor γ^t [13]. En el MDP de tokens, con una sola recompensa final, todas las $\widehat{Q}_t$ valen $r(x,y)$ y la proposición no aporta nada; su utilidad aparece con recompensas intermedias, como la penalización KL por token del tema 3.

2.5 Línea base y varianza

Lema 2.9 (La puntuación tiene media nula). *Para todo estado s, $\mathbb{E}_{a\sim\pi_\theta(\cdot|s)}\big[\nabla_\theta \log \pi_\theta(a \mid s)\big] = 0$.*

Proposición 2.10 (Línea base). *Sea $b(s)$ cualquier función del estado que no dependa de la acción. Entonces*

$$\nabla_\theta J(\theta) = \mathbb{E}_{p_\theta}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\big(\widehat{Q}_t - b(s_t)\big)\Big].$$

La línea base no cambia la esperanza, pero sí la varianza. En el ejemplo 2.7, la varianza total (traza de la matriz de covarianzas) del estimador de una muestra $(r(a)-b)(e_a-\pi)$ vale 0,889 con $b=0$ y 0,222 con $b=J=1$. Si se suma 10 a todas las recompensas, con $b=0$ sube a 80,9, mientras que con $b=J$ sigue siendo 0,222: la línea base hace el método insensible al origen arbitrario de la escala de recompensas. La clase 5 de CS285 observa que la recompensa media no es la línea base óptima, aunque sí una muy buena [12]; la proposición siguiente da la óptima.

Proposición 2.11 (Línea base de varianza mínima). *En un bandido con* $g(a) = \nabla_\theta \log \pi_\theta(a)$*, la varianza total del estimador* $(r(a)-b)\,g(a)$ *con* $a \sim \pi_\theta$ *es mínima para*

$$b^* = \frac{\mathbb{E}\big[r(a)\,\|g(a)\|^2\big]}{\mathbb{E}\big[\|g(a)\|^2\big]}.$$

2.6 Ventaja, crítico y estimación generalizada de la ventaja

La línea base natural en un MDP es $b(s_t) = V^\pi(s_t)$. Entonces $\widehat{Q}_t - V^\pi(s_t)$ es una estimación de la ventaja $A^\pi(s_t,a_t)$ de la definición 1.5, y el gradiente queda $\mathbb{E}[\sum_t \nabla \log \pi_\theta(a_t \mid s_t)\, A^\pi(s_t,a_t)]$, salvo los factores de descuento. Como V^π es desconocida, se aproxima con una red V_ϕ, el *crítico*, entrenada por regresión sobre los retornos observados. La política es el *actor*: de ahí el nombre de métodos actor-crítico [22, §13.5].

Definición 2.13 (Error de diferencia temporal y GAE). Dado un crítico V, el *error de diferencia temporal* es $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$, con $V(s_T)=0$ en el estado terminal. Para $\lambda \in [0,1]$, la *estimación generalizada de la ventaja* es [17]

$$\widehat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l\, \delta_{t+l},$$

que satisface la recursión $\widehat{A}_t = \delta_t + \gamma\lambda\, \widehat{A}_{t+1}$, con $\widehat{A}_T = 0$.

Proposición 2.14 (Casos extremos de GAE). *Con* $\lambda = 0$*,* $\widehat{A}_t = \delta_t$*. Con* $\lambda = 1$*,* $\widehat{A}_t = \sum_{l\ge 0} \gamma^l r_{t+l+1} - V(s_t)$*, el retorno observado menos la línea base. Si* $V = V^\pi$*, entonces* $\mathbb{E}[\delta_t \mid s_t, a_t] = A^\pi(s_t,a_t)$*.*

El parámetro λ regula el compromiso entre sesgo y varianza: con $\lambda=0$ se usa solo un paso de recompensa real y se confía en el crítico (poca varianza, sesgo si V es inexacto); con $\lambda=1$ se usa el retorno completo (sin sesgo, mucha varianza) [12]. PPO calcula sus ventajas con GAE.

Tema 3

Optimización proximal y regularización KL

REINFORCE usa cada lote de muestras para un único paso de gradiente. Este tema estudia cómo reutilizar las muestras sin alejarse demasiado de la política que las generó, mediante el muestreo de importancia y el recorte de PPO [18], y cómo se impide que la política se aleje del modelo de partida mediante una penalización de Kullback–Leibler. La parte de PPO sigue la clase 9 de CS285 [12]; la forma cerrada de la política óptima procede del apéndice A.1 de Rafailov *et al.* [16], y el objetivo con KL, de la §3.5 de InstructGPT [15].

3.1 Muestreo de importancia y objetivo sustituto

Proposición 3.1 (Muestreo de importancia). *Sean p y q distribuciones sobre un conjunto finito con $q(z) > 0$ siempre que $p(z) > 0$. Para toda función f,*

$$\mathbb{E}_{z\sim p}[f(z)] = \mathbb{E}_{z\sim q}\Big[\frac{p(z)}{q(z)}\, f(z)\Big].$$

Sea π_{old} la política con la que se generaron las muestras. Para una pareja estado-acción se define el *cociente de importancia*

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}.$$

El *objetivo sustituto* $L(\theta) = \mathbb{E}_{\pi_{\text{old}}}[\rho_t(\theta)\,\widehat{A}_t]$ estima, con muestras de π_{old}, la mejora local de la política π_θ; la aproximación consiste en ignorar que la distribución de estados también cambia con θ, lo que es aceptable si π_θ y π_{old} están próximas [12].

Proposición 3.2. $\nabla_\theta \rho_t(\theta) = \rho_t(\theta)\,\nabla_\theta \log \pi_\theta(a_t \mid s_t)$. *En particular, en $\theta = \theta_{\text{old}}$, $\nabla_\theta L = \mathbb{E}_{\pi_{\text{old}}}[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\,\widehat{A}_t]$, el gradiente de política con ventaja del tema 2.*

Optimizar L durante varios pasos con las mismas muestras es tentador, pero si ρ_t se aleja mucho de 1 la estimación pierde validez y su varianza crece. TRPO resuelve el problema con una restricción explícita sobre la divergencia KL entre π_{old} y π_θ; PPO lo resuelve con un recorte, mucho más sencillo de implementar [12, 18].

3.2 El objetivo recortado de PPO

Definición 3.3 (Objetivo recortado). Sea $\varepsilon > 0$ (típicamente $\varepsilon = 0{,}2$) y sea $\operatorname{clip}(\rho, 1-\varepsilon, 1+\varepsilon) = \min\{\max\{\rho, 1-\varepsilon\}, 1+\varepsilon\}$. El objetivo de PPO es [18, §3]

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\big(\rho_t(\theta)\widehat{A}_t,\ \operatorname{clip}(\rho_t(\theta), 1-\varepsilon, 1+\varepsilon)\,\widehat{A}_t\big)\Big].$$

Proposición 3.4 (Análisis por casos). *Para un término con ventaja A y cociente ρ, sea $\ell(\rho) = \min(\rho A, \mathrm{clip}(\rho)A)$.*

(i) $\ell(\rho) \le \rho A$ *siempre: el recorte nunca aumenta el objetivo.*

(ii) Si $A > 0$, $\ell(\rho) = \min(\rho, 1+\varepsilon)\,A$*: el término deja de crecer, y su derivada respecto a* ρ *se anula, cuando* $\rho > 1+\varepsilon$.

(iii) Si $A < 0$, $\ell(\rho) = \max(\rho, 1-\varepsilon)\,A$*: la derivada se anula cuando* $\rho < 1-\varepsilon$.

ρ	A	ρA	$\mathrm{clip}(\rho)A$	ℓ	Lectura
1,5	+2	3,0	2,4	2,4	acción buena que ya subió: sin incentivo para subir más
0,5	+2	1,0	1,6	1,0	acción buena que bajó: el gradiente la corrige
1,5	−1	−1,5	−1,2	−1,5	acción mala que subió: se penaliza entera
0,5	−1	−0,5	−0,8	−0,8	acción mala que ya bajó: gradiente nulo

Cuadro 3.1: Los cuatro casos del objetivo recortado con $\varepsilon = 0{,}2$.

El mínimo convierte L^{CLIP} en una cota *pesimista* del objetivo sustituto: el recorte elimina el incentivo para alejarse más de ε en la dirección favorable, pero nunca impide corregir un movimiento desfavorable (cuadro 3.1). En la práctica, PPO combina L^{CLIP} con una pérdida cuadrática para el crítico y una bonificación de entropía cuando ambos comparten parámetros, y el artículo describe también una variante con penalización KL adaptativa respecto a π_{old} [18, §4].

3.3 Penalización KL respecto a un modelo de referencia

En el ajuste por refuerzo de modelos de lenguaje no se maximiza solo la recompensa. Sea π_{ref} el modelo de partida congelado (el modelo SFT). El objetivo regularizado es [15]

$$\begin{aligned} &\max_{\pi}\ \mathbb{E}_x\big[J_\beta(\pi; x)\big], \\ &J_\beta(\pi; x) = \mathbb{E}_{y\sim\pi(\cdot|x)}[r(x,y)] - \beta\, D_{\mathrm{KL}}\big(\pi(\cdot \mid x)\,\|\,\pi_{\mathrm{ref}}(\cdot \mid x)\big), \end{aligned} \tag{3.1}$$

con $\beta > 0$.

La penalización cumple tres funciones: impide que la política explote errores del modelo de recompensa, que solo es fiable cerca de los datos con que se entrenó (tema 6); conserva la fluidez y el conocimiento del preentrenamiento, y mantiene la diversidad, ya que sin ella el óptimo es una política determinista que solo emite la respuesta de máxima recompensa.

Proposición 3.5 (KL de secuencias como suma por tokens). *Sean π y π_{ref} modelos autorregresivos sobre respuestas. Entonces*

$$\begin{aligned} D_{\mathrm{KL}}\big(\pi(\cdot \mid x)\,\|\,\pi_{\mathrm{ref}}(\cdot \mid x)\big) &= \mathbb{E}_{y\sim\pi}\Big[\sum_{t=1}^{T} \log \frac{\pi(y_t \mid x, y_{<t})}{\pi_{\mathrm{ref}}(y_t \mid x, y_{<t})}\Big] \\ &= \mathbb{E}_{y\sim\pi}\Big[\sum_{t=1}^{T} D_{\mathrm{KL}}\big(\pi(\cdot \mid s_{t-1})\,\|\,\pi_{\mathrm{ref}}(\cdot \mid s_{t-1})\big)\Big], \end{aligned}$$

con $s_{t-1} = (x, y_{<t})$.

La proposición justifica la implementación habitual: a la recompensa de cada token muestreado se le resta $\beta \log\big(\pi_\theta(y_t \mid s_{t-1})/\pi_{\text{ref}}(y_t \mid s_{t-1})\big)$, cuyo valor esperado acumulado es la KL. Cada término individual puede ser negativo; solo su esperanza es no negativa (ejercicio 3.5).

3.4 La política óptima en forma cerrada

Teorema 3.7 (Política óptima con penalización KL). *Para cada x, el objetivo (3.1) sobre las distribuciones $\pi(\cdot \mid x)$ alcanza su máximo, de forma única, en*

$$\pi^*(y \mid x) = \frac{1}{Z(x)}\,\pi_{\text{ref}}(y \mid x)\exp\Big(\frac{r(x,y)}{\beta}\Big), \qquad Z(x) = \sum_y \pi_{\text{ref}}(y \mid x)\, e^{r(x,y)/\beta},$$

y el valor máximo es $\beta \log Z(x)$.

β	π^*	$\mathbb{E}_{\pi^*}[r]$	$D_{\text{KL}}(\pi^* \,\|\, \pi_{\text{ref}})$
∞ (referencia)	(0,500; 0,300; 0,200)	1,100	0
2	(0,448; 0,443; 0,109)	1,335	0,058
1	(0,360; 0,587; 0,053)	1,534	0,205
0,5	(0,182; 0,808; 0,010)	1,798	0,587

Cuadro 3.2: Política óptima del ejemplo 3.8 en función de β.

Observación 3.9 (Distribución de Boltzmann y energía libre). Con $E(y) = -r(y)$ y temperatura β, la política óptima es $\pi^* \propto \pi_{\text{ref}}\, e^{-E/\beta}$: una distribución de Boltzmann (la softmax con temperatura del módulo 1) con π_{ref} como medida de fondo. El objetivo cambiado de signo, $\mathbb{E}_\pi[E] + \beta\, D_{\text{KL}}(\pi \,\|\, \pi_{\text{ref}})$, es una energía libre, y la demostración del teorema 3.7 es el principio variacional de Gibbs: el mínimo vale $-\beta \log Z$. Cuando $\beta \to 0$, π^* se concentra en las respuestas de máxima recompensa; cuando $\beta \to \infty$, $\pi^* \to \pi_{\text{ref}}$ (ejercicio 3.6).

El teorema 3.7 no puede aplicarse directamente a un LLM, porque $Z(x)$ es una suma sobre las $|\mathcal{V}|^T$ respuestas (observación 1.10). Por eso RLHF optimiza (3.1) con PPO. El tema 5 muestra que la forma cerrada sí puede aprovecharse si se trabaja con diferencias de recompensas, en las que $Z(x)$ se cancela.

Tema 4

Preferencias, modelo de recompensa y RLHF

Las personas no asignan con fiabilidad una nota absoluta a una respuesta, pero sí deciden con bastante consistencia cuál de dos respuestas prefieren. Este tema presenta el modelo de comparaciones por pares de Bradley y Terry [2], su estimación por máxima verosimilitud y su uso para entrenar un modelo de recompensa a partir de preferencias humanas, como en Christiano *et al.* [3], Stiennon *et al.* [21] y Ouyang *et al.* [15]. Termina con el procedimiento completo de RLHF de InstructGPT. La parte de aprendizaje de recompensas sigue además la tarea 3 de CS234 [5].

4.1 El modelo de Bradley–Terry

Definición 4.1 (Modelo de Bradley–Terry). Sean n objetos con puntuaciones latentes $r_1, \dots, r_n \in \mathbb{R}$. El *modelo de Bradley–Terry* postula que, al comparar i con j, se prefiere i con probabilidad

$$P(i \succ j) = \frac{e^{r_i}}{e^{r_i} + e^{r_j}} = \sigma(r_i - r_j),$$

y que las comparaciones son independientes entre sí.

La segunda igualdad se obtiene dividiendo numerador y denominador entre e^{r_i}. El modelo es una regresión logística (módulo 8) cuyo predictor es la *diferencia* de puntuaciones.

Proposición 4.2 (Propiedades). *(i)* $P(i \succ j) + P(j \succ i) = 1$, *y* $P(i \succ j) = \tfrac{1}{2}$ *si y solo si* $r_i = r_j$.

(ii) Las probabilidades no cambian si se suma una misma constante c a todas las puntuaciones; las puntuaciones solo están determinadas salvo una traslación.

(iii) Las razones de probabilidades *se multiplican:* $\dfrac{P(i \succ j)}{P(j \succ i)} \cdot \dfrac{P(j \succ k)}{P(k \succ j)} = \dfrac{P(i \succ k)}{P(k \succ i)}$.

La propiedad (iii) es una forma de transitividad: si i gana a j con razón 3:1 y j a k con razón 2:1, el modelo predice que i gana a k con razón 6:1, es decir, con probabilidad $6/7$.

Definición 4.3 (Modelo de Plackett–Luce). Para ordenar K objetos, el modelo de Plackett–Luce elige el primero con probabilidad proporcional a e^{r_i}, el segundo entre los restantes con la misma regla, y así sucesivamente:

$$P(i_1 \succ i_2 \succ \dots \succ i_K) = \prod_{k=1}^{K} \frac{e^{r_{i_k}}}{\sum_{m \geq k} e^{r_{i_m}}}.$$

Con $K = 2$ se reduce al modelo de Bradley–Terry.

4.2 Estimación por máxima verosimilitud

Proposición 4.4 (Pérdida del modelo de recompensa). *Sea* $\mathcal{D} = \{(x^{(m)}, y_w^{(m)}, y_l^{(m)})\}_{m=1}^{M}$ *un conjunto de comparaciones en las que* y_w *fue preferida a* y_l*, y sea* $r_\phi(x, y)$ *una recompensa parametrizada. Bajo el modelo de Bradley–Terry, el estimador de máxima verosimilitud de* ϕ *minimiza*

$$\mathcal{L}_{\mathrm{RM}}(\phi) = -\frac{1}{M}\sum_{m=1}^{M} \log \sigma\big(r_\phi(x^{(m)}, y_w^{(m)}) - r_\phi(x^{(m)}, y_l^{(m)})\big).$$

Con $z = r_\phi(x, y_w) - r_\phi(x, y_l)$*, el gradiente de cada término es* $-(1-\sigma(z))\big(\nabla_\phi r_\phi(x, y_w) - \nabla_\phi r_\phi(x, y_l)\big)$.

El factor $1-\sigma(z) = \sigma(-z)$ es la probabilidad que el modelo asigna al orden *equivocado*: los pares ya bien ordenados apenas contribuyen y los mal ordenados empujan con fuerza.

Proposición 4.6 (Estimador con dos objetos). *Si en* n *comparaciones entre* a *y* b *se prefiere* a *en* n_a *ocasiones, con* $0 < n_a < n$*, el estimador de máxima verosimilitud de la diferencia es* $\widehat{r_a - r_b} = \log\big(n_a/(n-n_a)\big)$*, y la probabilidad estimada es la frecuencia observada* n_a/n.

Si $n_a = n$, la verosimilitud crece sin cota cuando $z \to \infty$ y el estimador no existe: un modelo de recompensa sobreajustado puede separar las preferencias de entrenamiento con márgenes arbitrariamente grandes. En la práctica lo impiden la regularización y la parada temprana.

4.3 Modelos de recompensa neuronales

En el ajuste de LLM, $r_\phi(x, y)$ es un Transformer, normalmente inicializado desde el modelo SFT, en el que la capa de salida sobre el vocabulario se sustituye por una cabeza lineal que produce un escalar a partir del estado oculto del último token [21, 15]. Tres detalles merecen mención.

– *Comparaciones de segmentos.* Christiano *et al.* [3] aplicaron Bradley–Terry a pares de fragmentos de trayectoria en tareas de control, con la recompensa de un fragmento igual a la suma de recompensas por paso predichas (ejercicio 4.3).

– *Ordenaciones.* En InstructGPT, los anotadores ordenaban entre $K = 4$ y $K = 9$ respuestas a la misma instrucción. Cada ordenación produce $\binom{K}{2}$ comparaciones, que se tratan en un mismo lote y se ponderan por $1/\binom{K}{2}$ para que las instrucciones con más respuestas no dominen [15, §3.5].

– *Normalización.* Por la proposición 4.2(ii), la recompensa aprendida está definida salvo una constante; se suele trasladar para que las demostraciones de referencia tengan recompensa media nula antes del RL [21, 15].

4.4 El procedimiento RLHF de InstructGPT

El procedimiento de Ouyang *et al.* [15, §3.5] consta de tres etapas (figura 4.1):

1. *Ajuste supervisado*: el modelo preentrenado se ajusta con demostraciones escritas por anotadores (módulo 20).

2. *Modelo de recompensa*: se recogen ordenaciones de respuestas del modelo SFT y se entrena r_ϕ con la pérdida de la proposición 4.4. En el artículo, los modelos de recompensa tienen $6 \cdot 10^9$ parámetros.

3. *Optimización por refuerzo*: se maximiza $\mathbb{E}[r_\phi(x, y)] - \beta\, D_{\mathrm{KL}}(\pi_\theta \,\|\, \pi_{\mathrm{ref}})$ con PPO (tema 3). La variante PPO-ptx añade un término $\gamma\, \mathbb{E}_{x \sim \mathcal{D}_{\mathrm{pre}}}[\log \pi_\theta(x)]$ de verosimilitud sobre datos de preentrenamiento para reducir la pérdida de rendimiento en tareas clásicas.

Con este procedimiento, las respuestas de un modelo de $1{,}3 \cdot 10^9$ parámetros fueron preferidas por los evaluadores a las de GPT-3 de $1{,}75 \cdot 10^{11}$ parámetros [15]. El coste práctico es alto: durante la tercera etapa conviven en memoria la política, la referencia, el modelo de recompensa y el crítico de PPO, cuatro redes del tamaño de un LLM. Esta es la principal motivación práctica de DPO (tema 5).

Tema 5

Optimización directa de preferencias (DPO)

La optimización directa de preferencias (DPO) de Rafailov *et al.* [16] elimina el modelo de recompensa explícito y el bucle de refuerzo de RLHF. La idea es invertir la forma cerrada del teorema 3.7 para expresar la recompensa en función de la política y sustituirla en el modelo de Bradley–Terry, donde la función de partición se cancela. Este tema reproduce la derivación completa, los resultados de equivalencia de la §5.1 del artículo y el gradiente de la §4. El material se complementa con la clase invitada sobre DPO del curso CS234 [6] y con la tarea 5 de CS336 [9].

5.1 De la política óptima a la recompensa

Lema 5.1 (Reparametrización). *Sea π_r^* la política óptima del objetivo (3.1) para la recompensa r y la referencia π_{ref}, y supóngase $\pi_{\text{ref}}(y \mid x) > 0$. Entonces, para todo (x, y),*

$$r(x, y) = \beta \log \frac{\pi_r^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x).$$

El término incómodo, $\beta \log Z(x)$, depende solo de la instrucción x y no de la respuesta. Todo lo que sigue explota este hecho.

5.2 Clases de equivalencia de recompensas

Definición 5.2 (Recompensas equivalentes). Dos recompensas r y r' son *equivalentes* si existe una función f tal que $r'(x, y) = r(x, y) + f(x)$ para todo (x, y). Es una relación de equivalencia; sus clases se llaman *clases de recompensas*.

Lema 5.3 (Las recompensas equivalentes inducen las mismas preferencias). *Si r y r' son equivalentes, inducen la misma distribución de preferencias de Bradley–Terry, y también de Plackett–Luce, sobre las respuestas a cada instrucción.*

Lema 5.4 (Las recompensas equivalentes inducen la misma política óptima). *Si r y r' son equivalentes, $\pi_{r'}^* = \pi_r^*$.*

Teorema 5.5 (Representación de las clases de recompensas). *Toda clase de recompensas contiene un representante de la forma $\hat{r}(x, y) = \beta \log\big(\pi(y \mid x)/\pi_{\text{ref}}(y \mid x)\big)$ para alguna política π, y esa política es única: es la política óptima común de la clase.*

El teorema 1 de [16] enuncia este resultado: parametrizar las recompensas como $\beta \log(\pi/\pi_{\text{ref}})$ no pierde generalidad, porque toda clase de recompensas compatible con el modelo de pre-

ferencias puede escribirse así. Es el sentido preciso del subtítulo del artículo: el modelo de lenguaje es, en secreto, un modelo de recompensa.

5.3 La pérdida DPO

Proposición 5.6 (Preferencias en función de la política óptima). *Si las preferencias siguen el modelo de Bradley–Terry con recompensa r y $\pi^* = \pi_r^*$, entonces*

$$P(y_w \succ y_l \mid x) = \sigma\Big(\beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\Big).$$

Definición 5.7 (Pérdida DPO). Sea π_θ una política parametrizada y $\hat{r}_\theta(x, y) = \beta \log\big(\pi_\theta(y \mid x)/\pi_{\text{ref}}(y \mid x)\big)$ su *recompensa implícita*. La pérdida DPO sobre un conjunto de comparaciones $\mathcal{D}$ es

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\Big[\log \sigma\Big(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\Big)\Big].$$

La derivación completa se resume en cuatro pasos: (1) el objetivo con KL tiene la solución cerrada del teorema 3.7; (2) se despeja la recompensa (lema 5.1); (3) se sustituye en Bradley–Terry y se cancela $Z(x)$ (proposición 5.6); (4) se reemplaza π^* por π_θ y se maximiza la verosimilitud de las preferencias. El paso (4) es legítimo por el teorema 5.5: ajustar π_θ equivale a ajustar una recompensa dentro de una familia que, sin restricciones de capacidad, contiene todas las clases. Con otras palabras, $\mathcal{L}_{\text{DPO}}$ es la pérdida $\mathcal{L}_{\text{RM}}$ de la proposición 4.4 con r_ϕ sustituida por $\hat{r}_\theta$, y si $\hat{r}_\theta$ minimiza $\mathcal{L}_{\text{RM}}$, entonces π_θ es la política óptima del objetivo con KL para esa recompensa.

En la implementación, $\log \pi_\theta(y \mid x) = \sum_t \log \pi_\theta(y_t \mid x, y_{<t})$ se obtiene sumando las log-probabilidades por token de la respuesta (proposición 1.9), y las de π_{ref} se calculan una vez con el modelo congelado.

Proposición 5.8 (Gradiente de DPO). *Sea $h = \hat{r}_\theta(x, y_w) - \hat{r}_\theta(x, y_l)$. Entonces*

$$\nabla_\theta \mathcal{L}_{\text{DPO}} = -\beta\, \mathbb{E}_{\mathcal{D}}\Big[\sigma(-h)\big(\nabla_\theta \log \pi_\theta(y_w \mid x) - \nabla_\theta \log \pi_\theta(y_l \mid x)\big)\Big].$$

El gradiente aumenta la log-probabilidad de la respuesta preferida y reduce la de la rechazada. El peso $\sigma(-h)$ es la probabilidad que la recompensa implícita asigna al orden equivocado: es grande cuando el modelo ordena mal el par y casi nulo cuando ya lo ordena bien. Rafailov *et al.* observan que sin este peso el método degenera [16, §4].

5.4 DPO frente a RLHF con PPO

	RLHF con PPO	DPO
Redes en memoria	4 (política, referencia, recompensa, crítico)	2 (política, referencia)
Muestreo al entrenar	sí (*on-policy*)	no (datos fijos, *offline*)
Pérdida	objetivo recortado y penalización KL	clasificación binaria
Óptimo teórico	π^* del objetivo con KL	π^* del objetivo con KL

Cuadro 5.1: Comparación entre RLHF con PPO y DPO.

Ambos métodos persiguen el mismo óptimo (cuadro 5.1), pero no son equivalentes en la práctica. DPO solo ve las respuestas del conjunto de datos, generadas en general por otra política, y no explora; PPO evalúa con el modelo de recompensa las respuestas que la política genera en cada momento. Además, DPO controla solo la diferencia h: puede reducir a la vez las probabilidades de ambas respuestas siempre que la rechazada baje más, y puede sobreoptimizar su recompensa implícita, por ejemplo favoreciendo respuestas más largas (tema 6). El apéndice A.3 de [16] extiende la pérdida a ordenaciones completas con el modelo de Plackett–Luce.

Tema 6

Retroalimentación de IA, recompensas verificables y sobreoptimización

Los temas anteriores suponen que las preferencias proceden de personas y que la recompensa es fiable. Este tema estudia tres desarrollos que cuestionan esas hipótesis: la sustitución de parte de la supervisión humana por retroalimentación generada por un modelo según una lista de principios, Constitutional AI [1]; el uso de verificadores automáticos como recompensa, con el algoritmo GRPO [19, 4] y su análisis en la clase 17 y la tarea 5 de CS336 [7, 8], y la sobreoptimización de recompensas imperfectas, medida por Gao, Schulman y Hilton [10].

6.1 Constitutional AI y retroalimentación de IA

Constitutional AI (Bai *et al.*, Anthropic, 2022) [1] entrena un asistente inofensivo sin etiquetas humanas sobre inocuidad: la supervisión humana se concentra en redactar una lista de principios en lenguaje natural, la *constitución*. El método tiene dos fases (figura 6.1).

1. *Fase supervisada (SL-CAI).* Un modelo útil responde a instrucciones problemáticas. Después se le pide criticar su propia respuesta según un principio elegido al azar y reescribirla; el proceso puede repetirse. El modelo se ajusta con SFT sobre las revisiones finales.

2. *Fase de refuerzo (RL-CAI).* El modelo de la fase 1 genera pares de respuestas. Un modelo de retroalimentación decide cuál cumple mejor un principio, en algunos experimentos razonando paso a paso. Con esas preferencias generadas por IA se entrena un modelo de preferencias de Bradley–Terry (tema 4) y se optimiza la política contra él con RL y penalización KL (tema 3). Es lo que se denomina RLAIF, *RL from AI feedback*.

Las matemáticas son las de los temas 3 y 4; lo que cambia es quién genera las comparaciones. En el artículo, las preferencias de utilidad siguen procediendo de personas y las de inocuidad, del modelo [1]. La ventaja es doble: el método escala con mucha menos anotación humana, y los valores que se optimizan quedan explícitos y pueden auditarse. Anthropic ha descrito públicamente Constitutional AI como parte del entrenamiento de Claude; la constitución concreta de cada versión y los detalles actuales de su entrenamiento solo se conocen en la medida en que se han publicado.

6.2 Recompensas verificables

En matemáticas o programación no hace falta aprender la recompensa: se puede comprobar la respuesta.

Definición 6.1 (Recompensa verificable). Una *recompensa verificable* es una función $r(x, y) \in \{0, 1\}$ calculada por un programa: vale 1 si la respuesta final extraída de y coincide con la solución de referencia o si el código de y supera unas pruebas, y 0 en otro caso. El RL con recompensas de este tipo se denomina RLVR (*reinforcement learning with verifiable rewards*), término introducido en Tülu 3 [11].

Como solo se premia el resultado, el modelo es libre de escribir cualquier texto intermedio. DeepSeek-R1 [4] documenta que, al optimizar recompensas de exactitud y de formato con GRPO directamente sobre un modelo base, sin SFT previo (el modelo R1-Zero), las respuestas se alargan y aparecen cadenas de razonamiento extensas que aumentan la tasa de acierto. Sobre los modelos cerrados la información pública es escasa: OpenAI declaró que o1 (2024) se entrenó con RL a gran escala para razonar mediante cadenas de pensamiento sin publicar el algoritmo; Anthropic ofrece un modo de pensamiento extendido en Claude desde Claude 3.7 Sonnet (2025), y Google ofrece modos de razonamiento en Gemini. Los detalles de su entrenamiento no son públicos.

6.3 GRPO: optimización de política relativa al grupo

PPO necesita un crítico del tamaño del modelo para estimar $V(s_t)$. GRPO, introducido en DeepSeekMath [19, §4.1], lo sustituye por una línea base empírica: para cada instrucción se generan varias respuestas y cada una se compara con las demás.

Definición 6.2 (Ventaja relativa al grupo). Para una instrucción x se muestrean G respuestas $y^{(1)}, \dots, y^{(G)} \sim \pi_{\text{old}}(\cdot \mid x)$ con recompensas $r_1, \dots, r_G$. La ventaja de la respuesta i, asignada a todos sus tokens, es

$$\widehat{A}_i = \frac{r_i - \text{media}(r_1, \dots, r_G)}{\text{desv}(r_1, \dots, r_G) + \varepsilon_0},$$

con $\varepsilon_0 > 0$ pequeño para evitar la división por cero.

Definición 6.3 (Objetivo de GRPO). Con $\rho_{i,t} = \pi_\theta(y_t^{(i)} \mid x, y_{<t}^{(i)}) / \pi_{\text{old}}(y_t^{(i)} \mid x, y_{<t}^{(i)})$, el objetivo que se maximiza es [19, ec. (3)]

$$\frac{1}{G} \sum_{i=1}^{G} \frac{1}{|y^{(i)}|} \sum_{t=1}^{|y^{(i)}|} \Big[\min\big(\rho_{i,t} \widehat{A}_i,\ \text{clip}(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon) \widehat{A}_i\big) - \beta\, \widehat{D_{\text{KL}}}_{i,t} \Big],$$

donde $\widehat{D_{\text{KL}}}_{i,t} = u - \log u - 1$ con $u = \pi_{\text{ref}}(y_t^{(i)} \mid \cdot) / \pi_\theta(y_t^{(i)} \mid \cdot)$.

A diferencia de RLHF con PPO, la penalización KL no se resta de la recompensa sino que se añade directamente a la pérdida, y se estima con el estimador k_3 del ejercicio 3.5.

Proposición 6.4 (Propiedades del estimador k_3). *Sean p y q distribuciones con $q(y) > 0$ donde $p(y) > 0$, y sea $k_3(y) = u - \log u - 1$ con $u = q(y)/p(y)$. Entonces $k_3(y) \geq 0$ para todo y y $\mathbb{E}_{y \sim p}[k_3(y)] = D_{\text{KL}}(p \,\|\, q)$.*

Proposición 6.5 (Propiedades de las ventajas de grupo). *Con la desviación poblacional y $\varepsilon_0 = 0$, si no todas las recompensas son iguales, las ventajas de un grupo tienen media 0 y varianza 1. Si todas las recompensas son iguales, todas las ventajas son nulas y el grupo no aporta gradiente al término recortado.*

La media del grupo desempeña el papel de la línea base del tema 2. Con recompensas binarias y probabilidad de acierto p, un grupo de G respuestas no aporta señal con probabilidad $p^G + (1-p)^G$: las instrucciones que el modelo casi siempre resuelve, o casi nunca, desperdician cómputo (ejercicio 6.5).

Observación 6.7 (Sesgos de la normalización). Liu *et al.* [14] señalan dos sesgos en el objetivo de la definición 6.3. Dividir por la desviación del grupo da más peso a las instrucciones con poca variación de recompensa, y dividir por la longitud $|y^{(i)}|$ hace que, con ventaja negativa, cada token de una respuesta larga se penalice menos que uno de una corta, lo que favorece respuestas incorrectas largas. Su variante, Dr. GRPO, elimina ambas normalizaciones (ejercicios 6.3 y 6.4).

6.4 Sobreoptimización de la recompensa

Cuando se optimiza un indicador imperfecto, la política acaba encontrando formas de aumentarlo sin mejorar lo que el indicador pretendía medir. Es la ley de Goodhart aplicada al refuerzo, conocida como *reward hacking* (cuadro 6.1).

Indicador	Explotación típica
modelo de recompensa de utilidad	respuestas más largas, más listas y más halagos
pruebas unitarias	código que detecta la prueba y devuelve el valor esperado
satisfacción del usuario	dar la razón al usuario (adulación, *sycophancy*)
formato correcto	etiquetas impecables con contenido vacío

Cuadro 6.1: Ejemplos de explotación de recompensas.

Gao, Schulman y Hilton [10] midieron el fenómeno de forma controlada: un modelo de recompensa grande hace de recompensa «verdadera» y etiqueta los datos con los que se entrena un modelo de recompensa *proxy* más pequeño; después se optimiza la política contra el proxy y se mide la recompensa verdadera. Al aumentar la divergencia respecto a la política inicial, la recompensa proxy sigue creciendo, mientras que la verdadera crece, alcanza un máximo y decrece.

Definición 6.8 (Mejor de n). La política *mejor de n* (*best-of-n*) genera n respuestas independientes con la política inicial y devuelve la de mayor recompensa proxy.

Con $d = \sqrt{D_{\mathrm{KL}}(\pi \,\|\, \pi_{\mathrm{init}})}$, Gao *et al.* ajustan las formas funcionales

$$R_{\mathrm{bon}}(d) = d\,(\alpha_{\mathrm{bon}} - \beta_{\mathrm{bon}}\, d), \qquad R_{\mathrm{RL}}(d) = d\,(\alpha_{\mathrm{RL}} - \beta_{\mathrm{RL}} \log d)$$

para la ganancia de recompensa verdadera con mejor de n y con RL, con coeficientes que varían de forma suave con el tamaño del modelo de recompensa [10]. Para mejor de n, la divergencia se calcula analíticamente: $\mathrm{KL}_{\mathrm{bon}} = \log n - (n-1)/n$ (ejercicio 6.7).

Proposición 6.9 (Divergencia óptima). *R_{bon} alcanza su máximo en $d^* = \alpha_{\mathrm{bon}}/(2\beta_{\mathrm{bon}})$, con valor $\alpha_{\mathrm{bon}}^2/(4\beta_{\mathrm{bon}})$. R_{RL} alcanza su máximo en $d^* = e^{\alpha_{\mathrm{RL}}/\beta_{\mathrm{RL}} - 1}$, con valor $\beta_{\mathrm{RL}} d^*$.*

Existe, por tanto, una cantidad óptima de optimización, y más allá de ella la política empeora. Este es el argumento cuantitativo a favor de la penalización KL del tema 3: limita cuánto puede explotar la política los errores de r_ϕ. Otras defensas habituales son la parada temprana, el reentrenamiento del modelo de recompensa con muestras de la política actual, el uso de recompensas verificables cuando existen, la combinación de varios modelos de recompensa y la revisión manual de las muestras con recompensa anómalamente alta. DPO no es inmune: también puede sobreoptimizar su recompensa implícita.

Bibliografía

[1] Y. Bai, S. Kadavath, S. Kundu, A. Askell *et al.* Constitutional AI: Harmlessness from AI feedback. Prepublicación, Anthropic, 2022. https://arxiv.org/abs/2212.08073

[2] R. A. Bradley y M. E. Terry. Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.

[3] P. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg y D. Amodei. Deep reinforcement learning from human preferences. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. https://arxiv.org/abs/1706.03741

[4] DeepSeek-AI (D. Guo, D. Yang, H. Zhang, J. Song *et al.*). DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. Prepublicación, 2025. Publicado en *Nature*, 645:633–638, 2025. https://arxiv.org/abs/2501.12948

[5] E. Brunskill y equipo docente de CS234. CS234: Reinforcement Learning. Tarea 3. Stanford University, invierno de 2025. Secciones 1 «Reward engineering», 2 «Learning from preferences» y 3 «Direct preference optimization». https://web.stanford.edu/class/cs234/assignments/a3/hw3_questions.pdf

[6] E. Brunskill. CS234: Reinforcement Learning. Stanford University, primavera de 2024. Bloque «Policy Gradient» (clases 5–7) y clase 9, con la clase invitada sobre DPO de R. Rafailov, A. Sharma y E. Mitchell. https://web.stanford.edu/class/cs234/CS234Spr2024/modules.html

[7] P. Liang, T. Hashimoto *et al.* CS336: Language Modeling from Scratch. Stanford University, primavera de 2025. Clase 17: aprendizaje por refuerzo con recompensas verificables, gradiente de política y GRPO. https://github.com/stanford-cs336/spring2025-lectures

[8] Equipo docente de CS336. CS336: Language Modeling from Scratch. Tarea 5, «Alignment». Stanford University, primavera de 2025. §7.2 y §8. https://github.com/stanford-cs336/assignment5-alignment

[9] Equipo docente de CS336. CS336: Language Modeling from Scratch. Suplemento de la tarea 5, «Instruction Tuning and RLHF». Stanford University, primavera de 2026. §6.3. https://github.com/stanford-cs336/assignment5-alignment

[10] L. Gao, J. Schulman y J. Hilton. Scaling laws for reward model overoptimization. En *International Conference on Machine Learning (ICML)*, 2023. https://arxiv.org/abs/2210.10760

[11] N. Lambert, J. Morrison, V. Pyatkin, S. Huang, H. Ivison *et al.* Tülu 3: Pushing frontiers in open language model post-training. Prepublicación, Allen Institute for AI, 2024. https://arxiv.org/abs/2411.15124

[12] S. Levine. CS285: Deep Reinforcement Learning. University of California, Berkeley, otoño de 2023. Clases 4 «Introduction to Reinforcement Learning», 5 «Policy Gradients», 6 «Actor-Critic Algorithms» y 9 «Advanced Policy Gradients». https://rail.eecs.berkeley.edu/deeprlcourse-fa23/

[13] S. Levine y equipo docente de CS285. CS285: Deep Reinforcement Learning. Tarea 2, «Policy Gradients». University of California, Berkeley, otoño de 2023. §2 (repaso) y §8 (problemas de análisis). https://rail.eecs.berkeley.edu/deeprlcourse-fa23/

[14] Z. Liu, C. Chen, W. Li, P. Qi, T. Pang, C. Du, W. S. Lee y M. Lin. Understanding R1-Zero-like training: A critical perspective. Prepublicación, 2025. https://arxiv.org/abs/2503.20783

[15] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright *et al.* Training language models to follow instructions with human feedback. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. https://arxiv.org/abs/2203.02155

[16] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning y C. Finn. Direct preference optimization: Your language model is secretly a reward model. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. https://arxiv.org/abs/2305.18290

[17] J. Schulman, P. Moritz, S. Levine, M. I. Jordan y P. Abbeel. High-dimensional continuous control using generalized advantage estimation. En *International Conference on Learning Representations (ICLR)*, 2016. https://arxiv.org/abs/1506.02438

[18] J. Schulman, F. Wolski, P. Dhariwal, A. Radford y O. Klimov. Proximal policy optimization algorithms. Prepublicación, 2017. https://arxiv.org/abs/1707.06347

[19] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song *et al.* DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. Prepublicación, 2024. https://arxiv.org/abs/2402.03300

[20] D. Silver. UCL Course on Reinforcement Learning. University College London, 2015. Clase 2, «Markov Decision Processes». https://www.davidsilver.uk/teaching/

[21] N. Stiennon, L. Ouyang, J. Wu, D. M. Ziegler, R. Lowe, C. Voss, A. Radford, D. Amodei y P. Christiano. Learning to summarize from human feedback. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. https://arxiv.org/abs/2009.01325

[22] R. S. Sutton y A. G. Barto. *Reinforcement Learning: An Introduction*, 2.ª ed. MIT Press, 2018. Caps. 3 «Finite Markov Decision Processes» y 13 «Policy Gradient Methods». http://incompleteideas.net/book/the-book-2nd.html

[23] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992.